# Alpha Fitness

## Manual técnico y de administración

Documentación integral del proyecto

Edición 1.0
7 de octubre de 2026

Audiencia
Desarrollo y administración técnica

Revisión de la aplicación
`26b511f692c10fcb2e2d982ac94b407fbdc3b0fd`

Referencia para comprender la arquitectura, instalar la aplicación y administrar su configuración, datos y mantenimiento.

# Contenido

1 Identidad pila y alcance funcional ..... 3
2 Arquitectura implementada y entrypoints ..... 4
3 Instalación actualización y configuración ..... 5
Apartado 1 Preparación e instalación nueva ..... 5
Apartado 2 Actualización de una instalación existente ..... 6
Apartado 3 Configuración de servicios y entorno ..... 6
4 Autenticación autorización y rutas ..... 8
5 Contratos JSON y comportamiento del constructor ..... 13
6 Reglas de dominio integridad y límites ..... 15
7 Seguridad y manejo de datos ..... 16
8 Diccionario de datos final ..... 17
Apartado 1 Tablas y restricciones ..... 18
Apartado 2 Evolución del esquema y efectos operativos ..... 21
9 Despliegue y mantenimiento ..... 21
Apartado 1 Preparar una publicación ..... 21
Apartado 2 Aplicar el cambio y abrir el servicio ..... 22
Apartado 3 Mantenimiento ordinario ..... 23
10 Respaldo restauración y continuidad ..... 24
Apartado 1 Política y contenido de respaldo ..... 24
Apartado 2 Restaurar una copia de ensayo ..... 24
Apartado 3 Reversión y conservación histórica ..... 25
11 Pruebas diagnóstico y criterios de aceptación ..... 25
Apartado 1 Comprobaciones de una revisión ..... 25
Apartado 2 Evidencia archivada y comprobaciones pendientes ..... 26
Apartado 3 Diagnóstico de incidencias ..... 26
12 Fuentes trazabilidad y límites ..... 27
Apartado 1 Matriz de fuentes ..... 27
Apartado 2 Límites y diferencias relevantes ..... 28

**Edición:** 1.0

**Fecha:** 7 de octubre de 2026

**Revisión de aplicación:** `26b511f692c10fcb2e2d982ac94b407fbdc3b0fd`

**Audiencia:** responsables técnicos, administradores de infraestructura y mantenimiento.

**Estado:** contenido contrastado con fuentes de la revisión indicada; instalación, producción, integraciones externas y restauración requieren comprobación en el entorno destino.

Este manual describe la arquitectura implementada, requisitos, configuración, permisos, contratos internos, esquema de datos y procedimientos de operación de Alpha Fitness. Complementa el manual de usuario, que desarrolla las tareas de Administrador, Secretaria, Entrenador y Cliente desde la interfaz.

Las descripciones de comportamiento y estructura corresponden a hechos identificados en las fuentes. Los procedimientos de instalación, despliegue, respaldo y restauración son recomendaciones operativas derivadas de esas fuentes; no se presentan como ejecuciones efectuadas en producción. Se desconocen la configuración externa, el motor/versiones efectivos, los responsables del servicio y las políticas de conservación del despliegue.

# 1 Identidad pila y alcance funcional

El producto se presenta como **Alpha Fitness**; el directorio y paquete npm se llaman `alfa-fitness`. Es una aplicación web de gestión del gimnasio. Registra cobros presenciales de membresía y ventas de mostrador; los valores Efectivo, Tarjeta y Transferencia clasifican operaciones registradas, sin integración visible con una pasarela de pago. Las rutas son web y usan cookies/sesión; no hay una API REST pública versionada ni autenticación por token en el alcance inspeccionado.

| Componente | Requisito declarado / versión fijada | Fuente |
|---|---|---|
| PHP | `^8.2` | `composer.json:require`; requisito de Laravel y PhpSpreadsheet en `composer.lock` |
| Laravel | `^12.0` / `v12.69.3` | `composer.json`; `composer.lock:laravel/framework` |
| Socialite | `^5.29` / `v5.29.0` | `composer.lock:laravel/socialite` |
| Adaptador DomPDF | `^3.1.2` / `v3.1.2` | `composer.lock:barryvdh/laravel-dompdf` |
| DomPDF | `v3.1.6` | `composer.lock:dompdf/dompdf` |
| PhpSpreadsheet | `^5.10` / `5.10.0` | `composer.lock:phpoffice/phpspreadsheet` |
| Tinker | `^2.10.1` / `v2.11.1` | `composer.lock:laravel/tinker` |
| PHPUnit, desarrollo | `^11.5.50` / `11.5.56` | `composer.lock:phpunit/phpunit` |
| Vite | `^7.0.7` / `7.3.6` | `package.json`; `package-lock.json:node_modules/vite` |
| Laravel Vite Plugin | `^2.0.0` / `2.1.0` | `package-lock.json:node_modules/laravel-vite-plugin` |

| Componente | Requisito declarado / versión fijada | Fuente |
|---|---|---|
| Tailwind CSS y plugin Vite | `^4.0.0` / `4.3.3` | entradas correspondientes de `package-lock.json` |
| concurrently, desarrollo | `^9.0.1` / `9.2.4` | `package-lock.json:node_modules/concurrently` |
| Node.js | `^20.19.0 \|\| >=22.12.0` para Vite/plugin Laravel fijados | `package-lock.json:engines` |
| Composer | runtime API `^2.2` requerida por Laravel | `composer.lock:laravel/framework.require` |

El conjunto de extensiones PHP debe satisfacer Composer: ctype, filter, hash, mbstring, openssl, session, tokenizer, DOM, fileinfo, iconv, libxml, simplexml, XML, xmlreader, xmlwriter, ZIP, zlib y GD; añadir el driver PDO del motor seleccionado. OAuth necesita conectividad HTTPS y certificados válidos; cURL es la capacidad esperada en el entorno descrito por README. GD debe soportar WebP para las cargas de imágenes. Composer valida requisitos reales con `composer check-platform-reqs`. No usar `--ignore-platform-reqs` para declarar operativo un servidor incompleto.

SQLite y MySQL tienen configuración en `config/database.php`; este archivo también incluye MariaDB, PostgreSQL y SQL Server heredados de Laravel. Su presencia no demuestra pruebas de todos esos motores. No consta matriz certificada de SO o versión de servidor SQL. Los comandos siguientes se ejecutan desde la raíz del proyecto. En Windows pueden utilizarse desde PowerShell; PHP, Composer y Node.js deben estar en PATH o invocarse mediante su ruta instalada. No se exige XAMPP como componente de producción.

# 2 Arquitectura implementada y entrypoints

| Capa / ubicación | Responsabilidad y fuente principal |
|---|---|
| `public/index.php` | Entrada HTTP: mantenimiento, autoload Composer, arranque y `handleRequest` |
| `artisan` | Entrada CLI: arranque de la misma aplicación y `handleCommand` |
| `bootstrap/app.php` | Registra `routes/web.php`, `routes/console.php`, salud `/up`, middleware web y campo sensible `password_actual` que no debe reflashearse |
| `routes/web.php` | Contrato de navegación, verbos, nombres, autenticación, gates y throttles |
| `app/Http/Controllers` | Validación de solicitudes, autorización adicional, consultas y coordinación transaccional |
| `app/Services` | Registro de asistencia, analítica, PDF, Excel, imágenes e invitaciones |
| `app/Models` | Eloquent, relaciones, casts, identidad, propiedad y estados calculados |
| `app/Support` | Matriz `Acceso`, preferencias `Apariencia`, conversiones monetarias `Dinero` |
| `resources/views` | HTML Blade, formularios, presentación y JavaScript específico de rutinas |
| `resources/css/app.css`, `resources/js/app.js` | Entradas Vite declaradas en `vite.config.js`; recursos generados en `public/build` |

| Capa / ubicación | Responsabilidad y fuente principal |
|---|---|
| `database/migrations`, `database/seeders` | Evolución del esquema y catálogos iniciales; seeders no crean cuentas |
| `config` | Configuración desde entorno; su lectura no revela valores reales de `.env` |
| `tests`, `phpunit.xml`, `tools` | Pruebas PHPUnit y utilidades de QA/optimización; no constituyen CI configurada |

El flujo normal es solicitud HTTP → middleware web de Laravel y middleware añadidos → route binding/autenticación/gate → controlador → servicio/modelos/DB → vista, redirect, JSON o descarga. La solución es un monolito web; no se identifican repositorios de dominio, microservicios ni aplicaciones móviles separadas. `AppServiceProvider::boot` registra gates, límites por minuto y la huella de contraseña de sesión en eventos de login.

`Controller::actual` elige la identidad `cliente` si ambos guards coexistieran; en otro caso utiliza `web`. Los inicios de sesión cierran el otro guard. La separación depende conjuntamente del guard y del ID: `user_id` por sí solo no identifica al propietario de rutina, calificación o sesión. Esas relaciones no tienen FK hacia una sola tabla de usuarios y se filtran con `user_type`.

# 3 Instalación actualización y configuración

## Apartado 1 Preparación e instalación nueva

**Objetivo:** instalar una copia nueva con base dedicada. **Precondiciones:** revisión y locks disponibles, requisitos de la sección 1 y autorización del responsable sobre el entorno destino. Este procedimiento es una recomendación derivada del repositorio; no fue ejecutado durante la documentación. Conserve los locks y use `install`/`ci` para reproducir versiones.

1. Instalar dependencias: `composer install` y `npm ci`. Composer ejecuta descubrimiento de paquetes al regenerar autoload.
2. Solo en instalación nueva, copiar `.env.example` a `.env`: PowerShell `Copy-Item -LiteralPath .env.example -Destination .env`, verificando antes que no exista configuración que se deba conservar. Completar las variables necesarias.
3. Elegir base vacía y dedicada. SQLite: crear `database/database.sqlite` únicamente si no existe y definir `DB_CONNECTION=sqlite`; puede usarse `DB_DATABASE` con ruta absoluta. MySQL: crear previamente base y usuario de aplicación con permisos mínimos apropiados y configurar `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`. No asumir usuario root sin contraseña en producción.
4. Generar clave solo para esa instalación nueva: `php artisan key:generate`.
5. Aplicar esquema: `php artisan migrate --seed`. Revisar previamente seeders; crean catálogos de ejercicios/productos, mientras la migración de septiembre inserta planes. En actualizaciones de instalaciones existentes usar `php artisan migrate`, con respaldo, sin repetir seeders automáticamente.
6. Crear administrador inicial mediante `php artisan admin:create`; suministrar contraseña en prompt oculto, sin ponerla en argumentos o historial. El comando rechaza un administrador adicional y valida mínimo de 12 caracteres con mayúsculas/minúsculas, números y símbolos.

7. Compilar: `npm run build`; para desarrollo levantar `php artisan serve`. El frontend también puede usar `npm run dev` con Vite.

8. Verifique `/up`, carga de recursos, acceso del administrador y creación de una cuenta ficticia. Compruebe una invitación por correo en un entorno aislado antes de habilitar operación.

**Criterio de éxito:** dependencias compatibles, migraciones aplicadas, administrador creado sin duplicados, assets de la revisión disponibles y acceso funcional. Ante un fallo, conserve mensaje y estado, corrija requisitos o configuración y retome el paso afectado; no reinicialice la base para ocultar el error.

El alcance de `composer setup` requiere una instalación nueva: encadena instalación, creación de `.env`, generación de clave, creación SQLite, `migrate --force --seed`, npm y build. No corresponde a actualizar una instalación con datos. `composer dev` lanza servidor, escucha de colas, Pail y Vite mediante concurrently. `composer test` limpia caché de config antes de `artisan test`; también es una escritura de caché que requiere contexto adecuado. `npm run build` genera artefactos; no prueba los flujos funcionales.

## Apartado 2 Actualización de una instalación existente

**Objetivo:** aplicar una revisión conservando configuración, clave y registros. **Precondiciones:** respaldo coherente, restauración ensayada, revisión destino identificada y ventana acordada.

1. Identifique revisión desplegada, motor y migraciones ya aplicadas mediante `php artisan migrate:status`. Revise los efectos del apartado 2 del capítulo 8 y los cambios de fuentes/locks.

2. Prepare una copia de ensayo con el mismo esquema y conserve `.env`, `APP_KEY` e imágenes de la instalación. No copie secretos al repositorio.

3. Instale las dependencias de los locks de destino, compile recursos y ejecute la verificación correspondiente en ensayo. Use `composer install` y `npm ci`; no sustituya una actualización por `composer setup`.

4. Respalde el estado previo a la ventana, limite escrituras y publique la revisión preparada conforme a la sección 9.

5. Aplique solo migraciones pendientes con `php artisan migrate`; en producción se utiliza `php artisan migrate --force` después de revisar y autorizar sus efectos. No repita seeders automáticamente.

6. Regenere cachés, verifique acceso y operaciones de lectura, y comunique la invalidación de enlaces antiguos o revisión de asignaciones históricas cuando corresponda.

**Criterio de éxito:** datos y clave conservados, migraciones pendientes aplicadas, código y assets de la misma revisión y comprobaciones funcionales pertinentes correctas. Si falla una migración o comprobación, mantenga la ventana y determine si el esquema permite volver al código anterior. `migrate:rollback` no reemplaza un respaldo y puede eliminar datos.

## Apartado 3 Configuración de servicios y entorno

Complete las variables siguientes según el despliegue. Los valores reales de credenciales, host y responsables no se incluyen en este manual. Después de cambiar configuración, regenere su caché y compruebe la lectura efectiva antes de habilitar usuarios.

| Variables de entorno | Función / decisión de operación | Fuente |
|---|---|---|
| APP_NAME, APP_ENV, APP_DEBUG, APP_URL | Identidad, entorno, errores, enlaces absolutos; producción requiere URL HTTPS correcta y debug desactivado | config/app.php |
| APP_KEY, APP_PREVIOUS_KEYS | Cifrado y continuidad de descifrado; custodiar clave junto al plan de recuperación, no regenerar sobre instalación existente | config/app.php |
| APP_TIMEZONE, APP_LOCALE, APP_FALLBACK_LOCALE | Zona/idioma; ejemplo propone America/El_Salvador; fallback de config para zona es UTC | config/app.php, .env.example |
| DB_CONNECTION, DB_DATABASE, DB_URL, DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD, DB_FOREIGN_KEYS | Motor, conexión y restricciones; DB_URL puede prevalecer sobre parámetros por separado | config/database.php |
| SESSION_DRIVER, SESSION_LIFETIME, SESSION_ENCRYPT, SESSION_SECURE_COOKIE, SESSION_HTTP_ONLY, SESSION_SAME_SITE, SESSION_DOMAIN, SESSION_COOKIE | Sesión, caducidad y atributos de cookie; defaults database, 120 minutos, cifrada, HttpOnly y Lax; Secure por defecto en producción | config/session.php |
| CACHE_STORE, CACHE_PREFIX, DB_CACHE_CONNECTION, DB_CACHE_TABLE | Caché y rate limiting; default database | config/cache.php |
| QUEUE_CONNECTION, DB_QUEUE_CONNECTION, DB_QUEUE_TABLE, DB_QUEUE_RETRY_AFTER | Colas; default database con retry_after 90; su configuración no implica que todas las acciones se encolen | config/queue.php |
| MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_SCHEME, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM_ADDRESS, MAIL_FROM_NAME | Transporte y remitente; default log no entrega correo y puede registrar enlaces privados | config/mail.php |

| Variables de entorno | Función / decisión de operación | Fuente |
|---|---|---|
| GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI | OAuth de clientes; callback exacto /cliente/google/callback | config/services.php |
| PRIVACY_RESPONSIBLE_NAME, PRIVACY_CONTACT_EMAIL | Datos del responsable para política pública; completar antes de publicar | config/privacy.php |
| LOG_CHANNEL, LOG_STACK, LOG_LEVEL, LOG_DAILY_DAYS | Registro y retención de canal daily cuando se seleccione; proteger directorio/logs | config/logging.php |
| FILESYSTEM_DISK, BCRYPT_ROUNDS, APP_MAINTENANCE_DRIVER | Disco de framework, costo de hash y mantenimiento; ejemplo usa bcrypt 12 | config/filesystems.php, config hashing del framework, config/app.php |

No se proporcionan valores secretos. Los servicios AWS, Redis, Memcached, Slack, Postmark y Resend tienen entradas genéricas de configuración; su presencia no significa uso obligatorio ni integración funcional comprobada. Las imágenes de aplicación se escriben directamente en public/images, no mediante el disco por defecto. Cambiar zona horaria en una instalación con historial requiere analizar las fechas almacenadas; no convertirlas por suposición.

# 4 Autenticación autorización y rutas

config/auth.php define guards de sesión web→User y cliente→Cliente. Brokers separados: users en password_reset_tokens, clientes en cliente_password_reset_tokens; ambos expiran a 60 minutos y limitan regeneración a 60 segundos. Acceso::permite exige cuenta activa, rechaza roles desconocidos y define la siguiente matriz de **gates**, distinta de los accesos comunes que solo requieren autenticación:

| Gate | Administrador | Secretaria | Entrenador | Cliente |
|---|---|---|---|---|
| administrar | Sí | No | No | No |
| inventario | Sí | Sí | No | No |
| membresias | Sí | Sí | No | Sí |
| asistencia | Sí | Sí | No | No |
| analitica_financiera | Sí | No | No | No |
| asignar_rutinas | Sí | No | Sí | No |
| operaciones | No | Sí | No | No |
| progreso | No | No | No | Sí |

La tabla siguiente reconstruye nombres, verbos, controladores y barreras desde `routes/web.php`, complementadas con controles de los métodos indicados. A significa autenticación `auth:cliente,web` + `auth.session`; las rutas web incluyen CSRF y middleware de cuenta activa. Parámetros `{...}` usan route model binding. Métodos no listados como DELETE no deben suponerse disponibles.

| Ruta / método / nombre | Controlador::método o destino | Acceso efectivo |
|---|---|---|
| GET / | redirect login | Público |
| GET /privacidad privacidad | view privacidad | Público |
| GET /informacion informacion | view, planes activos por precio | Público |
| GET /login login; POST /login login.submit | LoginController::showLoginForm/login | Público; POST 5/min |
| POST /logout logout; POST /salir salir | LoginController::logout/salir | Cierre ambos guards / información conservando sesión |
| POST /cliente/login cliente.login.submit; POST /cliente/registro cliente.registro | ClienteLoginController::login/registrar | Público; 5/min cada ruta |
| POST /cliente/logout cliente.logout; POST /cliente/salir cliente.salir | ClienteLoginController::logout/salir | Cierre ambos guards / información |
| GET y POST /olvide-mi-contrasena password.request/email | EstablecerPasswordController::showForgotPasswordForm/sendResetLinkEmail | Público; POST 5/min |
| GET /establecer-contrasena/{token} password.reset; POST /establecer-contrasena password.update | EstablecerPasswordController::show/update | Público; POST 5/min |

| Ruta / método / nombre | Controlador::método o destino | Acceso efectivo |
|---|---|---|
| GET `/cliente/google` `cliente.google`; GET callback `cliente.google.callback` | ClienteLoginController::redirectToGoogle/handleGoogleCallback | Público; 20/min |
| GET `/dashboard` `dashboard` | DashboardController::index | web + auth.session |
| GET `/cliente/dashboard` `cliente.dashboard`; GET `/mi-asistencia` `cliente.asistencia` | DashboardController::index / AsistenciaController::miAsistencia | cliente + auth.session; historial propio |
| GET `/cuentas` `cuentas.index`; PATCH `/cuentas/{cliente}/banear/restaurar` `cuentas.banear/restaurar` | CuentasController::index/banear/restaurar | A + administrar |
| GET `/configuracion` `configuracion`; POST subrutas apariencia/perfil/password/personalizacion/avatar/google `configuracion.*` | ConfiguracionController::show/actualizarApariencia/actualizarPerfil/actualizarPassword/actualizarPersonalizacion/actualizarAvatar/vincularGoogle | A; password/google 5/min; Google solo cliente elegible y password actual |
| GET `/membresias` `membresias.index` | MembresiaController::index | A + membresias; clientes solo sus filas |
| POST `/membresias/planes` `planes.store`; PUT `/membresias/planes/{plan}` `planes.update`; PATCH toggle `planes.toggle` | PlanMembresiaController::store/update/toggle | A + administrar |
| POST `/membresias/solicitudes` `membresias.solicitar` | MembresiaController::solicitar | A + progreso, y cliente obligatorio |

| Ruta / método / nombre | Controlador::método o destino | Acceso efectivo |
|---|---|---|
| PATCH `/membresias/solicitudes/{solicitud}/activar` `membresias.activar` | MembresiaController::activar | A + operaciones, 60/min; Secretaria |
| PATCH solicitud cancelar `membresias.solicitudes.cancelar` | MembresiaController::cancelarSolicitud | A + propietario cliente o Secretaria |
| PATCH `/membresias/{membresia}/cancelar` `membresias.cancelar` | MembresiaController::cancelar | A + operaciones, Secretaria |
| POST `/membresias/{membresia}/pausa` `membresias.pausa.solicitar` | MembresiaController::solicitarPausa | A + propietario cliente o web Secretaria/Administrador |
| PATCH `/membresias/pausas/{pausa}/aprobar/rechazar` `membresias.pausa.aprobar/rechazar` | MembresiaController::aprobarPausa /rechazarPausa | A + operaciones; ruta limita a Secretaria |
| PATCH `/membresias/{membresia}/reanudar` `membresias.reanudar` | MembresiaController::reanudar | A + propietario cliente o Secretaria/Administrador |
| GET `/asistencia` `asistencia.index`; POST `/asistencia` `asistencia.store`; POST `/asistencia/salida` `asistencia.salida`; POST cerrar-huerfanas `asistencia.cerrar-huerfanas` | AsistenciaController::index/store/salida/cerrarHuerfanas | A + asistencia y guard web |
| GET `/asistencia/exportar` `asistencia.exportar` | AsistenciaController::exportar | A + asistencia; exportaciones 10/min |

| Ruta / método / nombre | Controlador::método o destino | Acceso efectivo |
|---|---|---|
| GET /entrenadores entrenadores.index; POST misma ruta entrenadores.store; PUT /{entrenador} entrenadores.update | EntrenadorController::index/store/update | A para directorio; escrituras administrar; alta 10/min |
| GET /productos productos.index; POST misma ruta productos.store; PUT /{producto} productos.update | ProductoController::index/store/update | A; escrituras inventario; sin permiso solo productos activos |
| GET/POST /ventas ventas.index/store; GET /{venta}/comprobante ventas.comprobante; POST /{venta}/enviar-correo ventas.enviar-correo | VentaController::index/store/comprobante/enviarCorreo | A + inventario; venta 60/min, correo 10/min |
| GET /analitica analitica.index; GET /analitica/pdf/excel analitica.pdf/excel | AnaliticaFinancieraController::index/exportarPdf/exportarExcel | A + analitica_financiera; descargas 10/min |
| GET/POST /progreso progreso.index/store; DELETE /{personalRecord} progreso.destroy | ProgresoController::index/store/destroy | A + progreso, cliente; borrado propio |
| GET /entrenador/clientes/{cliente}/progreso entrenador.cliente.progreso | ProgresoController::clienteProgreso | A; web Administrador o Entrenador con asignación por ID |
| GET /rankings y /estadisticas rankings.index/estadisticas.index | redirect progreso.index | A + progreso; no leaderboard público independiente |

| Ruta / método / nombre | Controlador::método o destino | Acceso efectivo |
|---|---|---|
| GET /ejercicios ejercicios.index; POST /{ejercicio}/calificar ejercicios.calificar | EjercicioController::index/calificar | A; califica ejercicio activo |
| POST /ejercicios ejercicios.store; PUT /{ejercicio} ejercicios.update; PATCH toggle ejercicios.toggle | EjercicioController::store/update/toggle | A + administrar |

En el grupo /entrenamientos, todos los nombres empiezan entrenamientos. y se dirigen a RutinaController. GET /→index; GET /historial→historial; POST /crear→crear; GET /catalogo/buscar→buscarEjercicios. GET/PUT/DELETE /{rutina}→editar/actualizar/eliminar; POST /{rutina}/asignar→asignar; GET /{rutina}/entrenar/{dia?}→entrenar; POST /{rutina}/finalizar→finalizarSesion; GET /{rutina}/imprimir→imprimir. Días: POST /{rutina}/dias→agregarDia (dias.crear), PUT/DELETE /dias/{dia}→renombrarDia/eliminarDia (dias.renombrar/eliminar). Ejercicios: POST /dias/{dia}/ejercicios→agregarEjercicio (ejercicios.crear), PUT /dias/{dia}/ejercicios/orden→reordenarEjercicios (ejercicios.reordenar), PUT/DELETE /ejercicios/{rutinaEjercicio}→actualizarEjercicio/eliminarEjercicio (ejercicios.actualizar/eliminar). Todos requieren A; datos de rutina/día/ejercicio exigen propietario con guard+ID. Asignar exige además gate asignar_rutinas y guard web. Crear rutina propia también está permitido a Secretaria. La ruta de salud /up se registra en bootstrap y no representa autenticación ni prueba de todos los servicios.

# 5 Contratos JSON y comportamiento del constructor

Estos endpoints son contratos internos consumidos por Blade/JavaScript y mantienen protección de sesión/CSRF. Enviar Accept: application/json, Content-Type: application/json y X-CSRF-TOKEN válido. En general éxito JSON devuelve 200; validación devuelve 422 con message y errors; acceso a recurso ajeno 403; binding inexistente 404; falta de autenticación JSON 401, token CSRF vencido 419 y throttle 429 según middleware Laravel. En navegación HTML predominan redirects con mensajes/errores de sesión. Los controladores no establecen respuesta 201 para estas operaciones.

| Operación | Payload / restricciones | Respuesta de éxito |
|---|---|---|
| PUT rutina | nombre requerido ≤100; objetivo requerido ≤60; nivel Principiante/Intermedio/Avanzado; dias_por_semana entero 1–7 | {ok:true,rutina:...} si wantsJson; alternativa HTML back |
| POST día / PUT día | Alta sin payload; renombrado con titulo requerido ≤100 | {dia:...} |

| Operación | Payload / restricciones | Respuesta de éxito |
|---|---|---|
| DELETE día / ejercicio | Propiedad comprobada; se compacta orden restante en transacción | `{ok:true}` |
| GET catálogo/buscar | q/grupo opcionales, strings ≤100; grupo Todos omite filtro | `{ejercicios:[...]}` con id, nombre, grupo_muscular, subgrupo, URLs y flags de imágenes |
| POST ejercicio del día | ejercicio_id activo existente | `{rutina_ejercicio:...}` con relación ejercicio cargada |
| PUT ejercicio del día | campos parciales: series 1–20, repeticiones texto ≤20, peso nullable 0–999, descanso_segundos 0–600 | `{rutina_ejercicio:...}` |
| PUT orden | array orden de IDs distintos, todos pertenecientes al día; incluir exactamente todos sus ejercicios | `{ok:true}` |
| POST finalizar | dia_id nullable perteneciente a rutina; duración 0–86400; series/total 0–500; notas nullable ≤500; sesion_uuid nullable UUID | `{ok:true,mensaje:...,sesion:...}` con expectsJson/ajax; alternativa redirect historial |
| POST calificar ejercicio | estrellas entero 1–5 | `{ok:true,mensaje:...,estrellas:...,promedio:...,total_votos:...}` |
| POST apariencia | design opcional de diseños permitidos; mode light/dark/custom; colors con claves primary/accent/background/surface/text y hex de 6 posiciones; contraste validado | `{appearance:...}` con expectsJson; alternativa back |

Fuentes: `RutinaController::{actualizar,agregarDia,renombrarDia,buscarEjercicios,agregarEjercicio,actualizarEjercicio,reordenarEjercicios,finalizarSesion}`, `EjercicioController::calificar`, `ConfiguracionController::actualizarApariencia`, `Apariencia::validar`.

El autosave de cabecera usa espera de 600 ms tras cambios y cola Promise para serializar envíos; una versión identifica el último estado. Campos de ejercicio se guardan en change, con input temporalmente deshabilitado y restitución del valor anterior ante error. `beforeunload` avisa cuando existe escritura pendiente o cabecera no guardada. Estos mecanismos son UX de guardado, no edición colaborativa ni resolución general de conflictos entre dispositivos. Fuente: `resources/views/entrenamientos/crear.blade.php:api,guardarCampo,guardarRutina`.

La finalización genera UUID al renderizar la página, bloquea doble click, mantiene UUID para reintentar desde esa página y solo muestra éxito tras respuesta `ok:true`. En servidor normaliza a minúsculas, usa firstOrCreate y verifica identidad y rutina; una colisión de UUID ajeno devuelve 403. No enviar UUID permite crear sesiones adicionales; refrescar la página genera un UUID distinto. Series completadas no pueden superar total_series. Inicio se calcula como hora de finalización menos duración informada; los checklists y tiempos vienen del navegador, no de instrumentación deportiva certificada. Borrar rutina conserva sesión con FK nula y nombres snapshot. Fuente: `RutinaController::finalizarSesion`,

`resources/views/entrenamientos/entrenar.blade.php:finalizarEntrenamiento` y migraciones de sesiones/UUID.

# 6 Reglas de dominio integridad y límites

**Identidad y personal.** El administrador único utiliza `admin_key='unico'` con índice unique; `User::booted` impide eliminarlo, desactivarlo o cambiar su rol mediante modelo. La migración de hardening rechaza bases con más de un Administrador antes de alterar cuentas. El alta de personal solo crea Entrenador o Secretaria con secreto aleatorio y `password_establecida=false`; acceso web no está habilitado hasta usar invitación. Desactivar conserva historial. La administración de clientes modifica activo y auditoría de ban; restaurar elimina los campos de ban actuales, no existe tabla histórica separada de todos los baneos. Fuentes: User, CreateAdminCommand, EntrenadorController, CuentasController.

**Membresías y cobros.** Solicitar copia nombre, precio, duración y condiciones del plan activo. Cambiar el plan después no reescribe esa solicitud. Bajo bloqueo de cliente se rechaza una solicitud pendiente duplicada del mismo plan. Activación bloquea cliente y solicitud, exige cuenta activa/solicitud pendiente, crea membresía y pago y resuelve solicitud en una transacción. Importe registrado es el confirmado por Secretaria, mínimo 0.01, máximo 999999 y hasta dos decimales; el controlador no exige igualdad con precio_acordado. Pago tiene uniques en membresia_id/solicitud_id; repetir activación rechaza sin cobrar dos veces.

La vigencia es inclusiva: fin=inicio+(duracion_dias−1). Si existe fin máximo no cancelado vigente/futuro, renovación comienza al día siguiente; si no, hoy. Cancelar conserva pago e historial, marca cancelada y rechaza pausas pendientes/aprobadas. No existe flujo de devolución monetaria ni borrado autorizado de pagos en las rutas revisadas. Fuentes: `MembresiaController::{solicitar,activar,cancelar}`, `PagoMembresia`, migración hardening.

**Pausas.** Solicitud de 3–30 días, motivo requerido ≤255, inicio hoy/futuro dentro de vigencia. Cliente propone pendiente; Secretaria o Administrador al solicitar la crea aprobada automáticamente. Cuenta debe estar activa, membresía no cancelada/vencida; máximo acumulado de 30 días por membresía contando aprobada/finalizada/reanudada_anticipada. No puede coexistir otra pausa pendiente/aprobada que siga vigente/programada. Aprobar verifica de nuevo bajo bloqueo cliente→membresía→pausa. Extensión de fin por días concedidos se rechaza si ocupa otra membresía no cancelada ya registrada; no desplaza otros períodos.

Reanudar pausa futura devuelve toda extensión y marca días=0; durante pausa calcula días efectivos inclusivos y quita extensión no usada. Estado visual En Pausa exige fecha actual entre límites y estado aprobada. Después de fin estimado deja de ser vigente por fechas; no se encontró tarea programada que cambie automáticamente aprobada a finalizada. El controlador acepta Administrador para aprobar/rechazar, pero gate operaciones de la ruta admite solo Secretaria: la autorización efectiva queda limitada por la ruta. Fuentes: MembresiaController, `Membresia::{getEstadoAttribute,diasPausadosAcumulados}`, `PausaMembresia::estaVigenteHoy`, routes/console.php.

**Asistencia.** `RegistroAsistencia::registrar` bloquea fila de cliente y realiza entrada/salida en transacción. Entrada exige activo, rechaza pausa aprobada actual sobre membresía no cancelada y rechaza visita abierta. No impone tener membresía vigente. Salida necesita visita abierta y puede cerrarla aunque la cuenta se haya desactivado. Visita abierta de más de 12 horas se autocierra al intentar nueva entrada con salida estimada entrada+2 horas; cierre masivo aplica misma regla y registra empleado. Son estimaciones operativas, no hora de salida observada. UI cliente consulta su historial; no hay ruta de autoentrada/autosalida del cliente actual. Dashboard aforo cuenta abiertas de últimas 12 horas, capacidad fija 80 y umbrales 50%/85%; pantalla

asistencia también muestra conteo de todas las abiertas, por lo que cifras pueden diferir. No se usa aforo para bloquear nuevas entradas. Fuentes: RegistroAsistencia, AsistenciaController, DashboardController::index.

**Ventas.** Precio y total se calculan en servidor, nunca se aceptan del carrito. `Dinero` transforma decimal de hasta dos posiciones a centavos enteros y devuelve string decimal. Hasta 100 items, cantidades 1–999; IDs repetidos se acumulan con máximo 999 por producto. En transacción se bloquea vendedor; si UUID ya existe, se devuelve venta del mismo vendedor incluso si el carrito cambió, y se rechaza vendedor distinto con 403. UUID se normaliza a minúsculas. Luego se bloquea cliente activo opcional y productos por ID ordenado, se valida disponibilidad/stock, crea venta y detalles y descuenta stock. Total máximo de esquema/código 99,999,999.99; productos admiten precio 0–999999, stock 0–999999. Métodos permitidos Efectivo/Tarjeta/Transferencia. Sin UUID no existe deduplicación general por contenido. Fuente: `VentaController::store`, `ProductoController::reglas`, `Dinero`, migración venta_uuid.

FK de ventas→empleado y detalles→producto es restrict tras migración de octubre; ya no borra venta o detalle al eliminar persona/producto. FK ventas→cliente es nullable/set null; eliminación directa de una venta aún cascada sus detalles en esquema, aunque no hay ruta de eliminación de venta. No confundir protección selectiva con inmutabilidad total de la base.

**Entrenamiento y progreso.** Rutina asignada es clon de días/ejercicios, se guarda nombre del asignador y su ID estable. Entrenador ve progreso solo si existe rutina de cliente asignada por su ID; el nombre histórico no otorga acceso y un ID nulo no acredita asignación. Administrador puede consultar progreso de clientes. PR del cliente: peso 0.5–999 kg, repeticiones 1–100, ejercicio activo y notas ≤255; mejores marcas se ordenan por peso×repeticiones con desempate por ID reciente. No se expone un endpoint actual de verificación por entrenador pese a campos verificado existentes. Calificaciones son una por ejercicio+usuario+guard, 1–5; listado obtiene promedio total, no limita votos a la semana pese al texto de populares. Fuentes: RutinaController::asignar, ProgresoController, EjercicioController, migración de asignado_por_id.

**Analítica.** Solo Administrador. Períodos hoy, ayer, esta_semana, semana_anterior, este_mes, mes_anterior, personalizado; por defecto mes actual. Fechas personalizadas Y-m-d, requeridas ambas, orden correcto y máximo 90 días inclusivos. Totales proceden de pagos_membresia.pagado_en y ventas.created_at, comparación con período inmediatamente anterior de igual duración. Importes se normalizan a centavos al límite DB; valores numéricos finales son contrato de presentación para gráficos/reportes. Incluye ingresos, conteos, ticket promedio, mezcla de ingresos, serie diaria con días sin movimiento, planes, productos y métodos; no calcula gastos, utilidad neta, impuestos ni conciliación bancaria. El dashboard usa un fallback a importes de membresías cuando pagos del mes no suman positivo; analítica no usa ese fallback, de modo que no deben prometerse cifras idénticas con historial legado. Fuentes: AnaliticaFinancieraService, DashboardController.

Los `lockForUpdate` y transacciones expresan diseño de serialización; pruebas SQLite no demuestran aislamiento y concurrencia efectiva de MySQL. El esquema no contiene unique parcial de una sola visita abierta ni constraint general de no solapamiento de membresías; las invariantes también dependen del uso de estos flujos. No escribir SQL administrativo directo para sortearlas.

# 7 Seguridad y manejo de datos

Passwords de modelos usan cast hashed; registro/cambio generan bcrypt/Hash. Reglas de altas/reset/cambio exigen 12 caracteres, mixedCase, numbers y symbols. Login recuerda solo si request remember es verdadero; OAuth/registro no activan remember automáticamente. Cada login regenera sesión

y cierra guard contrario. Logout invalida sesión y token CSRF, **no cierra sesión externa de Google**. Salir redirige a información conservando autenticación. Los efectos descritos corresponden al cuerpo vigente de los métodos, aunque algunos comentarios conservan descripciones anteriores.

`CuentaActiva` revisa ambos guards en cada solicitud web; si cuenta está inactiva o empleado sin contraseña establecida, cierra ambos y devuelve JSON403 o redirect login. `auth.session` y huella seeded al login ayudan a invalidar sesiones cuando cambia hash; restablecimiento/cambio rota remember_token. No se observó mecanismo de MFA ni confirmación obligatoria de correo local.

Google usa Socialite con estado de sesión; callback exige subject, email y email verificado boolean true. Busca por google_id y luego correo. No enlaza automáticamente cuenta local solo por coincidencia de correo: vinculación desde ajustes confirma password, conserva autorización de 5 minutos e identidad y exige correo coincidente. No almacena access/refresh tokens ni descarga avatar externo. Excepciones OAuth/mail registran solo clase de excepción, sin payload potencialmente sensible.

Middleware de encabezados agrega nosniff, DENY, none para cross-domain policies, Referrer-Policy, Permissions-Policy sin cámara/micrófono/geolocalización, COOP same-origin-allow-popups, CORP same-origin y CSP. CSP restringe origen de recursos, permite data/blob en imágenes y unsafe-inline en scripts/estilos actuales; local admite Vite HTTP/WebSocket en localhost. En production+isSecure agrega HSTS un año con subdominios y upgrade-insecure-requests. Páginas autenticadas/login/reset utilizan private,no-store; reset aplica no-referrer. Esto no demuestra TLS/proxy efectivo del servidor ni una CSP sin inline.

Throttles públicos 5/min en login/registro/recuperación/reset/cambio/vinculación, 20/min OAuth; named limits exportaciones=10/min y operaciones-financieras=60/min usan clase+ID autenticado o IP sin usuario. Debe conservarse store consistente para aplicar límites. Fuente: AppServiceProvider::boot y routes/web.php.

`ImagenSegura::guardar` admite destinos avatars/productos/ejercicios y prefijos restringidos; decodifica JPEG/PNG/WebP, máximo 16,777,216 píxeles, reescala hasta 640 px lado mayor en avatar y 1200 en otras imágenes, recodifica a WebP calidad82 con nombre UUID y elimina metadatos originales. Cuenta/avatar y personal/producto limitan 2MiB; ejercicios 8MiB por imagen. Perfil limita dimensiones4096²; personal/productos5000² además de límite de píxeles del servicio. Archivo nuevo se elimina si escritura DB falla; anterior se elimina tras reemplazo persistido. Avatares publicados son accesibles por URL sin autenticación: no subir documentos privados. Fuentes: ImagenSegura, ConfiguracionController, EntrenadorController, ProductoController, EjercicioController.

CSV asistencia incluye nombres/visitas/empleados, debe tratarse como información personal autorizada; BOM UTF8, streaming con lazy(500), escape ante caracteres que disparan fórmula. Excel financiero fija texto explícito para nombres y etiquetas, evitando ejecución de fórmula; cinco hojas Resumen, Ingresos por día, Membresías, Productos y Métodos de pago. PDF carta vertical usa DomPDF con remoto deshabilitado. Reportes financieros son agregados sin listado de clientes; registrar `reportes_financieros_logs` ocurre antes de generación, por lo que un log es intento registrado, no prueba de entrega exitosa. Comprobante de venta incluye relaciones personales y correo enviado a dirección validada que proporciona operador autorizado. Fuentes: AsistenciaController::exportar, ReporteExcelService, ReportePdfService, AnaliticaFinancieraController y VentaController.

# 8 Diccionario de datos final

Reconstrucción estática de **todas las migraciones en orden**, no extracción de una base privada. Notación: PK primary key; U unique; N nullable; FK→tabla.campo; C cascade on delete; R restrict on delete; S set null. `id`

generado por Blueprint id es BIGINT UNSIGNED autoincremental en semántica Laravel; tipos físicos dependen del motor. `string` sin longitud es VARCHAR(255); `remember_token` es VARCHAR(100) N. `timestamps` significa created_at/updated_at TIMESTAMP N. Boolean y JSON pueden materializarse distinto en SQLite; no asumir CHECK/enum si migración declara string. La tabla `migrations` es infraestructura del repositorio de migraciones de Laravel. Salvo las macros nullable `remember_token` y `timestamps` ya definidas, los campos sin N son no nullable; valores por defecto y restricciones se indican cuando están declarados. La notación compacta no convierte las validaciones del controlador en restricciones SQL.

## Apartado 1 Tablas y restricciones

| Tabla | Campos finales y restricciones | Fuente de creación + evolución |
|---|---|---|
| users | id PK; name string; email string U; email_verified_at timestamp N; password string; remember_token; timestamps; rol string default Secretaria; activo boolean default true; password_establecida boolean default true; admin_key string(20) N U; color_acento string default #facc15; avatar_piel string default claro; avatar_cabello string default corto; avatar_barba string default ninguna; avatar_atuendo string default basica; avatar_color_atuendo string default #1f2937; avatar string N; apariencia JSON N | 0001 users; 07-30 rol; 08-01 personalizacion users; 08-03 avatar; 09-09 harden; 09-10 appearance |
| clientes | id PK; nombre string; correo string U; password string N; google_id string N U; avatar string N; activo boolean true; remember_token; timestamps; mismos seis campos de personalización/defaults que users; apariencia JSON N; baneado_en timestamp N; baneado_por BIGINT UNSIGNED N FK users.id S; índice created_at | 07-30 create_clientes; 08-01 personalizacion clientes; 09-10 appearance; 09-14 ban audit |
| password_reset_tokens | email string PK; token string; created_at timestamp N | 0001 users |
| cliente_password_reset_tokens | email string PK; token string; created_at timestamp N | 10-07 isolate_client_password_reset_tokens |
| sessions | id string PK; user_id BIGINT UNSIGNED N índice sin FK; ip_address string(45) N; user_agent TEXT N; payload LONGTEXT; last_activity INTEGER índice | 0001 users |
| cache | key string PK; value MEDIUMTEXT; expiration INTEGER índice | 0001 cache |
| cache_locks | key string PK; owner string; expiration INTEGER índice | 0001 cache |
| jobs | id PK; queue string índice; payload LONGTEXT; attempts TINYINT UNSIGNED; reserved_at INTEGER UNSIGNED N; available_at/created_at INTEGER UNSIGNED | 0001 jobs |
| job_batches | id string PK; name string; total_jobs/pending_jobs/failed_jobs INTEGER; failed_job_ids LONGTEXT; options MEDIUMTEXT N; cancelled_at/finished_at INTEGER N; created_at INTEGER | 0001 jobs |
| failed_jobs | id PK; uuid string U; connection/queue/payload/exception TEXT/LONGTEXT (connection/queue TEXT, payload/exception LONGTEXT); failed_at timestamp default CURRENT_TIMESTAMP | 0001 jobs |

| Tabla | Campos finales y restricciones | Fuente de creación + evolución |
|---|---|---|
| ejercicios | id PK; nombre/grupo_muscular string; subgrupo string N; imagen/imagen_musculos string N; activo boolean true; timestamps. Nombre se valida unique en controlador, sin índice unique de esquema | 08-09 ejercicios; 08-10 imagen_musculos |
| ejercicio_calificaciones | id PK; ejercicio_id BIGINT UNSIGNED FK ejercicios.id C; user_id BIGINT UNSIGNED; user_type string; estrellas TINYINT UNSIGNED; timestamps; U(ejercicio_id,user_id,user_type) ejercicio_user_unique | 08-25 ejercicio_calificaciones |
| rutinas | id PK; user_id BIGINT UNSIGNED; user_type string; nombre/objetivo/nivel string; dias_por_semana TINYINT UNSIGNED default1; activa boolean true; asignado_por string N; asignado_por_id BIGINT UNSIGNED N FK users.id S; timestamps; índices(user_id,user_type), (asignado_por_id,user_type,user_id) | 08-09 rutinas; 09-22 asignado_por; 10-07 secure_routine_assignments_and_sessions |
| rutina_dias | id PK; rutina_id BIGINT UNSIGNED FK rutinas.id C; orden TINYINT UNSIGNED; titulo string; duracion_estimada_min/max SMALLINT UNSIGNED N; timestamps | 08-09 rutina_dias |
| rutina_ejercicios | id PK; rutina_dia_id BIGINT UNSIGNED FK rutina_dias.id C; ejercicio_id BIGINT UNSIGNED FK ejercicios.id C; orden TINYINT UNSIGNED; series TINYINT UNSIGNED default3; repeticiones string default8-10; peso DECIMAL(6,2) N; descanso_segundos SMALLINT UNSIGNED default60; timestamps | 08-09 rutina_ejercicios |
| personal_records | id PK; cliente_id FK clientes.id C; ejercicio_id FK ejercicios.id C; peso_kg DECIMAL(6,2); repeticiones TINYINT UNSIGNED default1; verificado boolean false; verificado_por BIGINT UNSIGNED N FK users.id S; fecha_verificacion timestamp N; notas string N; timestamps; índices(cliente_id,ejercicio_id,created_at), (ejercicio_id,verificado), (cliente_id,ejercicio_id) | 08-26 personal_records; 09-23 performance_indexes |
| asistencias | id PK; cliente_id FK clientes.id C; registrado_por/salida_registrada_por BIGINT UNSIGNED N FK users.id S; fecha_hora timestamp default CURRENT_TIMESTAMP; fecha_salida timestamp N; tipo_acceso string(30) default entrada; timestamps; índices(cliente_id,fecha_hora), fecha_hora, (fecha_salida,cliente_id), (cliente_id,fecha_hora,fecha_salida), además idx_asistencias_cliente_fecha(cliente_id,fecha_hora) e idx_asistencias_salida_fecha(fecha_salida,fecha_hora) | 08-27 asistencias + fecha_salida; 09-08 open_visits; 09-09 harden; 09-23 performance_indexes |
| productos | id PK; nombre string; precio DECIMAL(8,2); imagen string N; categoria string(100) N; stock INTEGER UNSIGNED default0; activo boolean true; orden TINYINT UNSIGNED default0; timestamps; índice(activo,orden) | 08-27 productos |
| planes_membresia | id PK; nombre string(100) U; precio DECIMAL(10,2); duracion_dias SMALLINT UNSIGNED; condiciones TEXT N; activo boolean true; timestamps | 09-09 harden |

| Tabla | Campos finales y restricciones | Fuente de creación + evolución |
|---|---|---|
| solicitudes_membresia | id PK; cliente_id FK clientes.id C; plan_id BIGINT UNSIGNED N FK planes_membresia.id S; plan_nombre string(100); precio_acordado DECIMAL(10,2); duracion_dias SMALLINT UNSIGNED; condiciones TEXT N; estado string(20) default pendiente; resuelta_en timestamp N; resuelta_por BIGINT UNSIGNED N FK users.id S; timestamps; índices(estado,created_at), (cliente_id,estado) | 09-09 harden |
| membresias | id PK; cliente_id FK clientes.id C; solicitud_id BIGINT UNSIGNED N U FK solicitudes_membresia.id S; plan string(100); importe DECIMAL(10,2); inicio/fin DATE; cancelada boolean false; activada_por BIGINT UNSIGNED N FK users.id S; timestamps; índice(cliente_id,cancelada,fin) | 09-08 membresias; 09-09 harden |
| pagos_membresia | id PK; membresia_id BIGINT UNSIGNED U FK membresias.id R; solicitud_id BIGINT UNSIGNED U FK solicitudes_membresia.id R; registrado_por BIGINT UNSIGNED N FK users.id S; importe DECIMAL(10,2); pagado_en timestamp; referencia string(100) N; timestamps | 09-09 harden |
| pausas_membresia | id PK; membresia_id FK membresias.id C; cliente_id FK clientes.id C; dias INTEGER UNSIGNED; motivo string(255); estado string(30) default pendiente; inicio_pausa/fin_pausa_estimada DATE; fecha_reanudacion DATE N; aprobada_por BIGINT UNSIGNED N FK users.id S; timestamps; índices(membresia_id,estado), (cliente_id,estado) | 09-23 pausas_membresia |
| ventas | id PK; user_id BIGINT UNSIGNED FK users.id R final; cliente_id BIGINT UNSIGNED N FK clientes.id S; total DECIMAL(10,2); metodo_pago string(50) default Efectivo; notas string(255) N; venta_uuid UUID N U; timestamps; índice created_at | 09-22 ventas; 10-07 preserve_financial_history; 10-07 add_sale_retry_uuid |
| detalle_ventas | id PK; venta_id FK ventas.id C; producto_id FK productos.id R final; cantidad INTEGER UNSIGNED; precio_unitario/subtotal DECIMAL(10,2); timestamps; índice(venta_id,producto_id) | 09-22 ventas; 10-07 preserve_financial_history |
| sesiones_entrenamiento | id PK; user_id BIGINT UNSIGNED; user_type string; rutina_id BIGINT UNSIGNED N FK rutinas.id S; rutina_nombre string; dia_id BIGINT UNSIGNED N FK rutina_dias.id S; dia_titulo string default Entrenamiento; iniciado_en/finalizado_en timestamp N; duracion_segundos/series_completadas/total_series INTEGER UNSIGNED default0; estado string default completado; notas TEXT N; sesion_uuid UUID N U; timestamps; índice(user_type,user_id) | 10-06 sesiones_entrenamiento; 10-07 secure_routine_assignments_and_sessions |
| reportes_financieros_logs | id PK; user_id FK users.id C; tipo string(20); fecha_inicio/fecha_fin DATE; created_at timestamp default CURRENT_TIMESTAMP; sin updated_at | 10-06 reportes_financieros_logs |
| migrations | id INTEGER UNSIGNED autoincremental PK; migration VARCHAR(255); batch INTEGER; sin FK ni timestamps | Infraestructura del repositorio de migraciones Laravel |

Todas las FK que se expresan sin tipo largo en la tabla proceden de `foreignId` (BIGINT UNSIGNED). Los dominios de strings estado/rol/user_type y rangos numéricos se validan mayormente en aplicación; no declarar ENUM/CHECK que no existen en migraciones. `asignado_por` y nombres de sesión son snapshots históricos. Precio de producto y precio_unitario/subtotal de venta se guardan aparte, por lo que cambiar precio actual no recalcula importes vendidos; el nombre de producto no tiene snapshot propio en detalle.

### Apartado 2 Evolución del esquema y efectos operativos

| Migración | Efecto comprobado y precaución de actualización |
|---|---|
| `2026_09_09_000000_harden_accounts_attendance_and_memberships.php` | Rechaza más de un Administrador, migra Recepcionista→Secretaria, cambia default rol, agrega guardas e infraestructura de solicitudes/pagos/planes; inserta tres planes. Resolver conflicto de múltiples admins antes de aplicar sin borrar cuentas arbitrariamente |
| `2026_09_10_000000_add_appearance_to_accounts.php` | Añade JSON de apariencia a ambos proveedores; configuración persistida por cuenta |
| `2026_09_14_000000_add_ban_audit_to_clientes.php` | Auditoría actual de ban y FK de actor |
| `2026_09_23_140000_add_performance_indexes.php` | Añade índices; no acredita rendimiento medido; conserva otro índice de asistencia con columnas equivalentes |
| `2026_10_07_100000_isolate_client_password_reset_tokens.php` | Crea tabla de tokens de clientes y borra tokens users previos por falta de identidad de proveedor; enlaces anteriores deben solicitarse de nuevo. down no recupera esos tokens |
| `2026_10_07_100000_preserve_financial_history.php` | Cambia deletes de ventas→users y detalles→productos a restrict; rollback devuelve cascadas y requiere revisión cuidadosa |
| `2026_10_07_110000_secure_routine_assignments_and_sessions.php` | Añade ID de asignador y UUID sesión; no infiere identidad de asignaciones antiguas por nombre. Verificar y reasignar por flujo autorizado |
| `2026_10_07_120000_add_sale_retry_uuid.php` | UUID nullable/unique para reintentos; registros históricos sin UUID siguen permitidos |

# 9 Despliegue y mantenimiento

### Apartado 1 Preparar una publicación

**Objetivo:** entregar una revisión coherente de backend, frontend y esquema. **Precondiciones:** entorno de ensayo verificado, configuración aprobada, respaldo restaurable y ventana de cambio acordada. Este proyecto no incluye un script de despliegue transaccional ni un pipeline CI propio.

1. Configure el servidor HTTP para servir exclusivamente `public/`. Proteja la raíz del repositorio, `.env`, logs, respaldos y dependencias privadas.

2. Prepare PHP y extensiones compatibles, base dedicada y permisos de escritura mínimos para `storage/`, `bootstrap/cache/` y directorios de imágenes. El código no necesita permisos generales de escritura sobre todo el proyecto.

3. Configure HTTPS, URL definitiva, SMTP, callback de Google cuando se utilice y responsables de privacidad. Mantenga `APP_ENV=production`, `APP_DEBUG=false`, cifrado de sesión, HttpOnly y SameSite=Lax; verifique cookies Secure y reconocimiento de HTTPS si existe proxy.

4. Construya la revisión con los comandos siguientes. No publican por sí mismos la aplicación ni prueban su funcionalidad:

```
composer install --no-dev --optimize-autoloader
composer check-platform-reqs
npm ci
npm run build
```

5. Conserve revisión y locks junto al artefacto. Si compila el frontend fuera del servidor, despliegue `public/build` completo y su `manifest.json`. Compruebe correspondencia de assets y revisión; un archivo `public/hot` de Vite de desarrollo no debe dirigir tráfico de producción.

**Criterio de éxito:** entorno compatible y artefacto reproducible con recursos de la misma revisión. Un error de Composer o build exige resolver requisitos o código antes de continuar; no suprima validaciones de plataforma.

## Apartado 2 Aplicar el cambio y abrir el servicio

1. Reserve la ventana y limite escrituras. `php artisan down` activa mantenimiento; confirme que los servicios y workers que utilice el despliegue no siguen generando escrituras incompatibles.

2. Obtenga el respaldo consistente del estado previo según la sección 10.

3. Publique el código y assets preparados conservando configuración, clave e imágenes. Revise migraciones incrementales antes de ejecutarlas.

4. Aplique pendientes y genere cachés:

```
php artisan migrate --force
php artisan optimize
```

5. Mientras continúa mantenimiento, compruebe estado de migraciones, generación de cachés, disponibilidad de archivos, errores de CLI y salud `/up`, que Laravel excluye del bloqueo de mantenimiento. Los recorridos de inicio de sesión y navegación autenticada reciben una respuesta de mantenimiento y no pueden validarse todavía.

6. Si utiliza trabajos encolados, gestione sus consumidores con el supervisor del servidor y reinícielos conforme a la configuración efectiva. `php artisan queue:restart` solicita recarga a workers persistentes; no instala un supervisor.

7. Tras superar los controles técnicos, termine mantenimiento con `php artisan up` para una reapertura controlada y ejecute inmediatamente las pruebas de humo: `/up`, recursos, acceso de ambos tipos de cuenta, permisos y lecturas pertinentes. La ruta `/up` demuestra respuesta de salud de Laravel; no acredita correo, OAuth ni todas las reglas del negocio.

8. Revise errores y compruebe las invitaciones/exportaciones previamente ensayadas con identidades ficticias en el entorno autorizado. Evite mensajes a usuarios reales. Si falla una prueba de humo, vuelva a `php artisan down`, conserve la evidencia y decida recuperación o retorno compatible antes de reabrir.

**Criterio de éxito:** versión publicada identificada, esquema compatible, cachés actuales, HTTP y assets disponibles y pruebas de humo pertinentes satisfactorias. Si falla un punto, mantenga el servicio controlado, registre la causa y decida restauración o retorno compatible. No ejecute automáticamente un rollback de migraciones.

## Apartado 3 Mantenimiento ordinario

| Área | Comprobación recomendada | Respuesta |
|---|---|---|
| Disponibilidad | Salud HTTP, errores de aplicación y conectividad DB | Identificar causa y alcance antes de reiniciar componentes. |
| Almacenamiento | Espacio disponible, directorios escribibles y tamaño de exports/logs | Liberar o ampliar según política de conservación autorizada. |
| Correo | Transporte, remitente y entrega de una prueba aislada | Corregir SMTP sin exponer contraseñas ni enlaces privados. |
| Google | URL/callback y acceso en entorno autorizado | Comprobar credenciales y estado de sesión; preservar alternativa local. |
| Datos | Visitas abiertas, cuentas activas, inventario y operaciones inconsistentes | Usar UI y flujos autorizados; evitar SQL que eluda invariantes. |
| Copias | Resultado, integridad y restauraciones periódicas | Aplicar controles de la sección 10. |
| Dependencias | Locks, avisos de seguridad y compatibilidad | Revisar cambios y ejecutar regresión antes de publicar. |

`routes/console.php` solo declara el comando de ejemplo `inspire`: no programa copias, cierre de visitas ni transición de pausas a finalizada. La vigencia de pausa depende de fechas. Añadir un cron no crea tareas que el proyecto no define.

`composer dev` lanza servidor, cola, Pail y Vite para desarrollo. Los mensajes de invitación y comprobante examinados usan envío síncrono; una cola activa no garantiza su entrega. Defina supervisión de workers solo cuando el despliegue tenga trabajos que la requieran.

Para sustituir arte original, `php tools/optimize_images.php` genera copias WebP preservando fuentes y escribe en `public/images`. Revise imágenes destino y capacidad GD antes de ejecutarlo. Las bajas ordinarias de personal y productos se realizan por desactivación para conservar relaciones financieras. Las utilidades de QA no deben ejecutarse contra producción por asumir que su nombre garantiza inocuidad.

# 10 Respaldo restauración y continuidad

## Apartado 1 Política y contenido de respaldo

**Objetivo:** recuperar datos, configuración y archivos de un mismo estado. **Precondiciones:** mecanismo consistente del motor y ubicación privada con acceso restringido. Frecuencia, retención, RPO y RTO no están definidos en el código; el responsable debe acordarlos y comprobarlos mediante ensayos.

El respaldo mínimo reúne base íntegra, referencia de revisión y locks, migraciones aplicadas e imágenes en `public/images/avatars`, `public/images/productos` y `public/images/ejercicios`, además del arte necesario. Custodie `APP_KEY` y configuración por separado con controles equivalentes; perder la clave puede impedir descifrar datos protegidos. No coloque copias en `public/` ni en Git. Los logs pueden contener información personal y enlaces privados cuando se utiliza el transporte de correo `log`.

1. Identifique motor, versión, tamaño, punto de copia y posibles escrituras concurrentes.
2. Detenga o coordine escrituras para obtener el estado consistente acordado. Seleccione el mecanismo de backup del motor; no hay un comando universal de copia implementado por la aplicación.
3. En SQLite, contemple transacciones y WAL; copiar solamente un archivo vivo no acredita consistencia. En MySQL, utilice un procedimiento compatible con el motor/tablas, versiones, charset, collation y claves foráneas.
4. Copie archivos vinculados al mismo punto de datos, resguarde configuración/clave y registre revisión, hora, alcance y checksum.
5. Compruebe integridad del archivo, posibilidad de lectura y disponibilidad de las claves en su custodia autorizada.

**Comprobaciones de aceptación del respaldo**

- Base y archivos pertenecen al mismo punto de recuperación; no faltan tablas o imágenes necesarias.
- Copia y configuración están fuera del área pública, con permisos y custodia definidos.
- Checksum, revisión, fecha y responsable de la copia quedan registrados sin secretos.
- La copia se puede abrir o importar en ensayo; crear un archivo sin probar su recuperación no acredita un respaldo utilizable.

## Apartado 2 Restaurar una copia de ensayo

**Objetivo:** demostrar recuperación antes de necesitarla en producción. **Precondiciones:** backup íntegro, clave correcta, revisión compatible y entorno aislado.

1. Prepare host/base y namespace independientes. Desactive correo real y callbacks de producción para evitar mensajes y accesos externos a cuentas reales.
2. Detenga escrituras del ensayo y restaure la base con el mecanismo del motor elegido. No importe sobre una base ajena ni sobre la única copia conservada.
3. Restaure imágenes y configuración compatibles; conserve la clave del respaldo sin ejecutar `key:generate`.
4. Publique el código/locks correspondientes al estado copiado. Ajuste URL y servicios de ensayo sin alterar el conjunto original.

5. Limpie y regenere cachés según esa configuración, inicie la aplicación y compruebe conteos, relaciones, imágenes, accesos y muestras de rutinas, pagos, ventas y detalles.

6. Compare los resultados con el inventario de la copia, registre duración efectiva y documente errores o pérdidas encontradas.

**Comprobaciones de aceptación de restauración**

- DB cargada y esquema compatible, con relaciones/FK consistentes y conteos esperados.
- Accesos y descifrado correctos con la clave conservada; imágenes de la copia accesibles.
- Muestras de membresía/pago y venta/detalle conservan importes y vínculos.
- Servicios del ensayo permanecen aislados; no se enviaron mensajes ni se escribió en producción.
- Tiempo y punto de recuperación medidos cumplen RTO/RPO acordados, o la desviación queda registrada para corregirla.

Ante fallos de FK, registros ausentes, imágenes faltantes o descifrado, mantenga aislada la copia y resuelva la causa. No publique una restauración parcial como completa. Para una recuperación real, repita el procedimiento sobre el destino expresamente autorizado y valide antes de reabrir escrituras.

### Apartado 3 Reversión y conservación histórica

Volver al código anterior solo es viable si su esquema y datos siguen siendo compatibles. `migrate:rollback` puede borrar tablas/columnas o restaurar borrados en cascada; `down` de aislamiento de tokens no recupera los enlaces eliminados. Ninguno de esos comandos reemplaza restaurar un backup comprobado.

No infiera identidades de entrenadores a partir de nombres duplicables ni rehaga cobros para completar historia. Las incidencias financieras requieren preservar registros y decidir la corrección con el responsable. Desactivar o banear mantiene información; no representa borrado ni cumple automáticamente una política de conservación.

## 11 Pruebas diagnóstico y criterios de aceptación

### Apartado 1 Comprobaciones de una revisión

Ejecute los siguientes controles sobre una copia de ensayo y registre directorio, revisión, toolchain, resultado y número de casos. Compruebe sus efectos antes de emplearlos sobre otro entorno: build y cache escriben artefactos; `composer test` limpia caché de configuración.

| Comando | Qué demuestra | Límite |
|---|---|---|
| `composer check-platform-reqs` | Extensiones y plataforma compatibles con dependencias instaladas | No prueba aplicación o servicios. |
| `php artisan route:list` | Rutas registradas por el arranque/configuración efectiva | No prueba permisos ni respuesta de todas las rutas. |
| `php artisan migrate:status` | Estado registrado de migraciones en la base elegida | No acredita integridad de todos los datos. |
| `php artisan test` | Casos realmente ejecutados en entorno de pruebas | Confirmar casos, aserciones y omisiones; salida 0 sola no basta. |

| Comando | Qué demuestra | Límite |
|---|---|---|
| `npm run build` | Compilación frontend y assets generados | No prueba interacción o responsive. |
| `php artisan view:cache` | Compilación de vistas Blade | Escribe caché y no prueba todos los datos presentados. |
| `vendor/bin/pint --test` | Convenciones PHP sin aplicar cambios | En Windows puede invocarse `php vendor/bin/pint --test`. |
| `composer audit` y `npm audit` | Avisos en dependencias según información consultada | Cambian con el tiempo y no prueban ausencia total de vulnerabilidades. |

La configuración `phpunit.xml` utiliza SQLite `:memory:`, caché y sesión `array`, correo `array` y cola `sync`. No se sustituye por una base privada para verificar documentación. Las pruebas existentes cubren módulos, autorización, recuperación, privacidad, integridad financiera, reintentos y límites. Los bloqueos/transacciones requieren comprobación de concurrencia en el motor del despliegue; SQLite en memoria no certifica ese comportamiento en MySQL.

## Apartado 2 Evidencia archivada y comprobaciones pendientes

El archivo `output/hardening/phpunit-publish.xml` registra **166 pruebas, 1095 aserciones, 0 errores, 0 fallos y 0 omitidas**. Se consultó como evidencia previa del proyecto, no se ejecutó una suite nueva al redactar este manual. Estos resultados no certifican SMTP, Google real, TLS/proxy, instalación limpia, restauración ni rendimiento productivo.

Para aceptar un despliegue, compruebe en su revisión/configuración efectiva: recursos servidos, guard correcto, acceso denegado por rol, cuenta inactiva, solicitud sin activación automática, cobro sin repetición, inventario y stock, rutina privada, progreso autorizado, exportaciones y cierre de sesión. Use datos ficticios y preserve la separación del entorno. Una modificación de fuentes, dependencias, esquema o configuración invalida las comprobaciones que dependan de lo cambiado.

## Apartado 3 Diagnóstico de incidencias

| Síntoma | Revisión técnica inicial | Criterio para resolver |
|---|---|---|
| Recursos ausentes | `manifest.json`, publicación completa, Vite hot y rutas del servidor | Assets de la revisión servidos sin referencia de desarrollo. |
| Error 419 | Cookies, sesión, CSRF y reconocimiento HTTPS | Formulario nuevo con sesión vigente funciona. |
| Error 403 | Gate, rol, estado activo y propiedad guard más ID | La operación autorizada funciona y la ajena sigue denegada. |
| Correo no entregado | Mailer efectivo, SMTP y remitente; el mensaje genérico no acredita envío | Prueba aislada recibida sin exposición de enlaces. |
| Acceso Google fallido | Estado de sesión, callback, correo verificado y autorización de vínculo | Identidad elegible retorna a la cuenta correcta. |

| Síntoma | Revisión técnica inicial | Criterio para resolver |
|---|---|---|
| Error de carga de imagen | GD/WebP, MIME, dimensiones, píxeles, permisos y espacio | Archivo válido recodificado y referencia DB persistida. |
| Cifras financieras distintas | Período/zona, pagos registrados y fallback legado del dashboard | Diferencia explicada por fuentes y datos, sin alterar historia arbitrariamente. |
| Error 500 | Log privado y clase/contexto del error, espacio, DB y permisos | Causa corregida y flujo afectado repetido correctamente. |

Restrinja logs y capturas: no incluya credenciales, enlaces de recuperación ni datos reales en informes generales. Registre el alcance y resultados de la corrección sin afirmar que una lectura estática equivale a una ejecución.

# 12 Fuentes trazabilidad y límites

## Apartado 1 Matriz de fuentes

Las rutas de archivo son relativas a la raíz del proyecto. Los nombres abreviados del diccionario remiten a las migraciones fechadas de `database/migrations`, en orden cronológico. La tabla `migrations` procede de infraestructura de Laravel, no de una migración de negocio.

| Secciones | Fuentes de autoridad | Comprobación documental |
|---|---|---|
| 1 y 3 | `composer.json`, `composer.lock`, `package.json`, `package-lock.json`, `.env.example`, archivos `config` | Requisitos, versiones fijadas, scripts y configuración contrastados. |
| 2 y 4 | `public/index.php`, `artisan`, `bootstrap/app.php`, `routes/web.php`, `routes/console.php`, `app/Providers/AppServiceProvider.php`, `app/Support/Acceso.php` | Entry points, rutas, guards, gates y límites contrastados. |
| 5 y entrenamiento | `app/Http/Controllers/RutinaController.php`, `resources/views/entrenamientos/crear.blade.php`, `entrenar.blade.php` | Payloads, respuestas, autosave, propietario y reintentos contrastados. |
| 6 operaciones | Controladores `MembresiaController`, `VentaController`, `AsistenciaController`, `ProgresoController`, `EjercicioController`; servicios `RegistroAsistencia`, `AnaliticaFinancieraService`; modelos de dominio | Reglas, transacciones, importes y límites contrastados. |

| Secciones | Fuentes de autoridad | Comprobación documental |
|---|---|---|
| 7 | Middleware `CuentaActiva` y `EncabezadosSeguros`; controladores Auth/Configuracion; servicios `ImagenSegura`, `Invitaciones`, `ReportePdfService`, `ReporteExcelService`; vista `privacidad` | Sesión, OAuth, archivos y exportaciones contrastados. |
| 8 | Todos los archivos de `database/migrations`, modelos Eloquent y repositorio de migraciones del framework | Esquema final reconstruido, nulabilidad, FK e índices identificados. |
| 9 y 10 | Scripts Composer/npm, config, rutas de consola y evolución del esquema | Procedimientos recomendados; no se operó producción ni restauró una base. |
| 11 | `phpunit.xml`, archivos `tests/Feature`, XML archivado | Fuente y resultados previos diferenciados de ejecución nueva. |

## Apartado 2 Límites y diferencias relevantes

El README atribuye auto registro de visitas a clientes; las rutas vigentes solo ofrecen consulta privada, y entrada/salida corresponden al personal autorizado. Algunos comentarios de login/logout describen persistencia automática o cierre de Google, pero el código usa el valor de `remember` y cierra sesiones locales. La pantalla de acceso actual no muestra casilla Recordarme.

El Administrador carece del gate `operaciones`; las rutas de aprobación/rechazo de pausa pendiente se reservan a Secretaria. Aunque solicitar/reanudar acepta Administrador en servidor, la interfaz no muestra sus botones de gestión. Cancelar membresía activada y anular pausa futura tienen acciones de servidor sin recorrido ordinario visible en esa pantalla. Estas diferencias no se convierten en instrucciones con controles inexistentes.

La política pública menciona avance temporal del entrenamiento; en la pantalla examinada solo consta persistencia explícita de la preferencia de sonido en `localStorage`. El checklist y tiempo en curso no deben considerarse recuperables tras recargar. Los campos de verificación de marcas existen en DB, sin endpoint actual de verificación por entrenador.

La instalación real de correo, Google, TLS/proxy, motor DB, backups y políticas no fue inspeccionada. Este manual no certifica seguridad absoluta, disponibilidad, rendimiento, cumplimiento legal ni descubrimiento de agentes en Antigravity. El contenido corresponde a la revisión indicada y debe revisarse cuando cambien sus fuentes.